Complete End-to-End Data Lineage & Database Table Mapping

This document provides a comprehensive architectural flow chart and data dictionary for every report, dashboard, and modal across all 8 modules of the **Bajaj MES Dashboard** system.

1. High-Level System Architecture Data Flow

The diagram below illustrates the end-to-end data pipeline from user interaction on the shop floor to the SQL Server database (`PPMS_BajajPant`) and back:

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph Client["Frontend Client (React 19 + Vite + TailwindCSS)"]
        UI["User Interface (Pages & Modals)"]
        Filter["StandardFilterBar (Period, Shift, Line, Station, Model, SKU)"]
        Hooks["Hooks (useReportFilters, useFilterOptions)"]
        Fetch["Fetch Client (HTTP GET with query parameters)"]
        State["React State (dbData, kpis, tableData)"]
        Render["Render Layer (StatCard, Recharts, DataTable, Modal, XLSX Export)"]
    end

    subgraph Server["Backend Express API Server (Node.js on Port 5000)"]
        Router["Express Router (/api/*)"]
        ParamParser["Parameter Normalizer (normalizeShift, computeDateRange)"]
        QueryBuilder["SQL Query & Stored Procedure Builder"]
        DBPool["MSSQL Connection Pool (Windows Auth / localhost)"]
        JSONRes["JSON Response Formatter ({ kpis, table, chartData })"]
    end

    subgraph Database["Microsoft SQL Server (Database: PPMS_BajajPant)"]
        SP["Stored Procedures (GV_ProdShiftPlan, DS_Dashboard_*)"]
        TransTables["Transactional Tables (Prod_EnginePlanExecution, Perf_Downtime, ...)"]
        ConfigTables["Configuration Tables (Config_Line, Config_Station, Config_Model, ...)"]
        MasterTables["Material & Stock Tables (Material_Stock, Config_PartVariant, Config_MaterialLocationMapping)"]
    end

    UI --> Filter
    Filter --> Hooks
    Hooks --> Fetch
    Fetch -->|HTTP GET ?period=Today&shift=Shift 1...| Router
    Router --> ParamParser
    ParamParser --> QueryBuilder
    QueryBuilder --> DBPool
    DBPool -->|Execute T-SQL / SP| Database
    Database -->|Recordsets| DBPool
    DBPool --> JSONRes
    JSONRes -->|JSON Payload| State
    State --> Render
    Render --> UI
```

2. Global Filter & Parameter Normalization Flow

Before any SQL query runs, user filter inputs pass through standard normalization logic:

SPECIFICATION / ARCHITECTURE

```
flowchart LR
    A["User Selects: Period (Today, Shift, Week, Month, Custom)"] --> B["computeDateRange()"]
    C["User Selects: Shift (Shift 1, Shift 2, Shift 3, All)"] --> D["normalizeShift()"]
    E["User Selects: Dropdowns (Line, Station, Model, SKU)"] --> F["Sanitize & Optional Filter"]

    B --> G["@StartDate, @EndDate"]
    D --> H["@Shift (Mapped to DB values '1','2','3' or 'A','B','C')"]
    F --> I["@Line, @Station, @Model, @SKU"]

    G & H & I --> J["Parameterized SQL Query / Stored Procedure"]
```

3. Module-by-Module Data Flow & Table Mappings

Module 1: Production Module

1.1 Architectural Flow Chart

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph UI_Layer["Production Reports"]
        P1["Production.jsx (Overview & Trend)"]
        P2["StraightPassReport.jsx (FTR & Rework)"]
    end

    subgraph API_Layer["Express API Endpoints"]
        E1["GET /api/dashboard/production"]
        E2["GET /api/production/plan"]
        E3["GET /api/production/straight-pass"]
    end

    subgraph DB_Layer["Database Tables & Procedures"]
        T1[("Prod_EnginePlanExecution")]
        T2[("Perf_Hourly_OLE")]
        T3[("Perf_Downtime")]
        T4[("Config_LossCategory")]
        T5[("Config_SubLossCategory")]
        T6[("Config_Line")]
        T7[("Config_SKU")]
        T8[("Config_Model")]
        T9[("Config_ModelFamily")]
        T10[("Prod_Engine_WIP")]
        T11[("Prod_Defect_Log")]
        SP1[["SP: DS_Dashboard_ProductionKPI"]]
        SP2[["SP: GV_ProdShiftPlan"]]
    end

    P1 --> E1
    P2 --> E3
    E1 --> SP1
    E1 --> T1
    E1 --> T2
    E1 --> T3
    T3 --- T4
    T3 --- T5
    E1 --> T6 & T7 & T8 & T9
    E2 --> SP2
    E3 --> T1 & T6 & T10 & T11 & T7 & T8
```

1.2 Data Dictionary & Table Cross-Reference

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination

Production Report (`Production.jsx`)	`/api/dashboard/production`	`DS_Dashboard_ProductionKPI` (SP)
		`Prod_EnginePlanExecution`
		`Perf_Hourly_OLE`
		`Config_Line`
		`Config_SKU`
		`Config_Model`
		`Config_ModelFamily`
			`E.LineID = L.LineID`
			`E.SKUID = S.SKUID`
			`S.ModelID = M.ModelID`
			`M.ModelFamilyID = F.ModelFamilyID`
				`PlanQty`, `ENGCompleted_Qty`, `ENGReworkOK_Qty`, `ENGMainLine_Qty`, `ENGMaterialHold_Qty`, `ENGQualityHold_Qty`, Hourly `PlannedQuantity`, `GoodQuantity`	`StatCard` (Plan, Actual, Shortfall, WIP, Rollover), Plan vs Actual Trend Chart, SKU Data Table

Straight Pass Report (`StraightPassReport.jsx`)	`/api/production/straight-pass`	`Prod_EnginePlanExecution`
		`Prod_Engine_WIP`
		`Prod_Defect_Log`
		`Config_Line`
		`Config_SKU`
		`Config_Model`
			`E.LineID = L.LineID`
			`E.SKUID = S.SKUID`
			`S.ModelID = M.ModelID`
			`E.EngineNo = W.EngineNo`
				Total Inspected, Straight Pass Qty, Rework Pass Qty, Rejected Qty, First-Time-Right (FTR) Yield %, Defect Descriptions	FTR % Gauge Card, Reworked Trend, Straight Pass vs Defect Breakdown Table

Production Shift Plan	`/api/production/plan`	`GV_ProdShiftPlan` (Official SP)	Param: `@ProdDate`, `@ProdShift`	Scheduled Model Codes, Planned Line Allocations, Shift Target Counts	Master Shift Production Schedule Table

Module 2: Performance Module (OEE / OLE & Downtime)

2.1 Architectural Flow Chart

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph UI_Layer["Performance Reports"]
        PF1["Performance.jsx (OLE / OEE Dashboard)"]
        PF2["DowntimeReport.jsx (Downtime Analysis)"]
    end

    subgraph API_Layer["Express API Endpoints"]
        E_PF1["GET /api/dashboard/performance"]
        E_PF2["GET /api/performance/downtime"]
    end

    subgraph DB_Layer["Database Tables & Procedures"]
        D1[("Perf_Hourly_OLE")]
        D2[("Perf_Downtime")]
        D3[("Prod_EnginePlanExecution")]
        D4[("Config_Line")]
        D5[("Config_Station")]
        D6[("Config_LossCategory")]
        D7[("Config_SubLossCategory")]
    end

    PF1 --> E_PF1
    PF2 --> E_PF2
    E_PF1 --> D1 & D2 & D3 & D4
    E_PF2 --> D2
    D2 --- D4
    D2 --- D5
    D2 --- D6
    D2 --- D7
```

2.2 Data Dictionary & Table Cross-Reference

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Performance (OLE/OEE) Dashboard (`Performance.jsx`)	`/api/dashboard/performance`	`Perf_Hourly_OLE` `Perf_Downtime` `Prod_EnginePlanExecution` `Config_Line`	`H.SubAsslyLineID = L.LineID` `D.SubAsslyLineID = L.LineID`	`OLE_Percentage`, `Availability_Percentage`, `Performance_Percentage`, `Quality_Percentage`, `TotalDT`, Operating Time	OLE/OEE Radial Gauges, Availability/Performance/Quality KPI Cards, Hourly OLE Trend Chart
Downtime Report (`DowntimeReport.jsx`)	`/api/performance/downtime`	`Perf_Downtime` `Config_Line` `Config_Station` `Config_LossCategory` `Config_SubLossCategory`	`D.SubAsslyLineID = L.LineID` `D.StationID = S.StationID` `D.LossID = LC.LossID` `D.SubLossID = SLC.SubLossID`	`StartTime`, `EndTime`, `TotalDT` (Minutes), `Reason`, `LossName`, `SubLossName`, `StationName`, `LineName`	Total Downtime Duration, Stoppage Incidents, Top Loss Pareto Chart, Downtime Log Grid

Module 3: Process Monitoring Module

3.1 Architectural Flow Chart

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph UI_Layer["Process Monitoring Reports"]
        PR1["PokaYokeReport.jsx (Interlocks)"]
        PR2["PokaYokeBypassReport.jsx (Bypass Audits)"]
        PR3["TorqueReport.jsx (Torque & Angle)"]
        PR4["ConveyorReport.jsx (Speed & JPH)"]
    end

    subgraph API_Layer["Express API Endpoints"]
        E_PR1["GET /api/process/pokayoke"]
        E_PR2["GET /api/process/bypass"]
        E_PR3["GET /api/process/torque"]
        E_PR4["GET /api/process/conveyor"]
    end

    subgraph DB_Layer["Database Tables & Procedures"]
        PM1[("Prod_EnginePlanExecution")]
        PM2[("Perf_Downtime")]
        PM3[("Config_Line")]
        PM4[("Config_Station")]
        PM5[("Config_User")]
        PM6[("Config_SKU")]
        PM7[("Config_Model")]
        PM8[("Prod_TorqueData_Log")]
        SP_CONV[["SP: DS_Dashboard_ProcessConveyor"]]
    end

    PR1 --> E_PR1
    PR2 --> E_PR2
    PR3 --> E_PR3
    PR4 --> E_PR4

    E_PR1 --> PM1 & PM2 & PM3 & PM4
    E_PR2 --> PM2 & PM3 & PM4 & PM5 & PM6 & PM7
    E_PR3 --> PM8 & PM6 & PM7
    E_PR4 --> SP_CONV
```

3.2 Data Dictionary & Table Cross-Reference

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Poka Yoke Report (`PokaYokeReport.jsx`)	`/api/process/pokayoke`	`Prod_EnginePlanExecution` `Perf_Downtime` `Config_Line` `Config_Station`	`D.SubAsslyLineID = L.LineID` `D.StationID = S.StationID`	Total Interlock Checks, Active Interlocks, Failed Checks, Station-wise Poka Yoke Status	Interlock OK/NOK Counter, Station Compliance Bar Chart, Real-time Sensor Interlock Grid
Poka Yoke Bypass Report (`PokaYokeBypassReport.jsx`)	`/api/process/bypass`	`Perf_Downtime` `Config_Line` `Config_Station` `Config_User` `Config_SKU`	`D.SubAsslyLineID = L.LineID` `D.StationID = S.StationID` `D.UserID = U.UserID`	Bypass Timestamp, Station ID, Reason / Authorization, Approver Name (`Config_User`), Duration	Total Bypass Events, Active Bypasses, High-risk Station Table, Authorized Bypass Log
Torque Report (`TorqueReport.jsx`)	`/api/process/torque`	`Prod_TorqueData_Log` `Config_SKU` `Config_Model`	`T.SKUID = S.SKUID` `S.ModelID = M.ModelID`	`SpindleNo`, `TargetTorque`, `ActualTorque`, `TargetAngle`, `ActualAngle`, `TorqueStatus` (OK/NOK), `AngleStatus`	Fastening Pass Rate %, Torque Distribution Curve, Tightening Angle vs Torque Scatter, Bolt-level Log
Conveyor Report (`ConveyorReport.jsx`)	`/api/process/conveyor`	`DS_Dashboard_ProcessConveyor` (SP)	Param: `@LineID`, `@ProdDate`	Conveyor Belt Speed (m/min), Drive Motor Current, JPH (Jobs Per Hour), Pitch Time, VFD Frequency	Line Speed Gauge, JPH Actual vs Target Trend, Motor Load Graph, Carrier Tracking Grid

Module 4: Track & Trace Module

4.1 Architectural Flow Chart

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph UI_Layer["Track & Trace Reports"]
        TR1["GenealogyReport.jsx (Engine Child BOM)"]
        TR2["WIPReport.jsx (Line Buffer Status)"]
        TR3["ReworkStatusReport.jsx (Rework Stations)"]
        TR4["EngineReworkReport.jsx (Execution Log)"]
    end

    subgraph API_Layer["Express API Endpoints"]
        E_TR1["GET /api/trace/genealogy"]
        E_TR2["GET /api/trace/wip"]
        E_TR3["GET /api/trace/rework"]
        E_TR4["GET /api/trace/engine-rework"]
    end

    subgraph DB_Layer["Database Tables & Procedures"]
        TT1[("Prod_Engine_Geneology")]
        TT2[("Prod_Engine_WIP")]
        TT3[("Prod_Defect_Log")]
        TT4[("Config_Station")]
        TT5[("Config_User")]
        TT6[("Config_SKU")]
        TT7[("Config_Model")]
        TT8[("Config_Line")]
        SP_WIP[["SP: DS_Dashboard_TrackTraceWIP"]]
    end

    TR1 --> E_TR1
    TR2 --> E_TR2
    TR3 --> E_TR3
    TR4 --> E_TR4

    E_TR1 --> TT1 & TT4 & TT5
    E_TR2 --> SP_WIP
    E_TR3 --> TT3 & TT2 & TT6 & TT7 & TT8
    E_TR4 --> TT3 & TT2 & TT6 & TT7
```

4.2 Data Dictionary & Table Cross-Reference

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Genealogy Report (`GenealogyReport.jsx`)	`/api/trace/genealogy`	`Prod_Engine_Geneology` `Config_Station` `Config_User`	`G.StationID = S.StationID` `G.UserID = U.UserID`	`EngineNo`, `PartBarcode`, `PartName`, `StationName`, `ScanTimestamp`, `OperatorName`, Batch Code	2D Barcode Search Bar, Engine BOM Hierarchy Tree, Child Part Traceability Grid
WIP Report (`WIPReport.jsx`)	`/api/trace/wip`	`DS_Dashboard_TrackTraceWIP` (SP)	Param: `@LineID`, `@ProdDate`	`StationID`, `StationName`, `EngineCount`, `BottleneckFlag`, Buffer Utilization %, Min/Max Dwell Time	Line Balance Flow Chart, Station WIP Heatmap, Bottleneck Warning Banner, Station WIP Table
Rework Status Report (`ReworkStatusReport.jsx`)	`/api/trace/rework`	`Prod_Defect_Log` `Prod_Engine_WIP` `Config_SKU` `Config_Model` `Config_Line`	`D.EngineNo = W.EngineNo` `W.SKUID = S.SKUID` `S.ModelID = M.ModelID` `W.LineID = L.LineID`	`EngineNo`, `DefectCode`, `DefectDescription`, `Status` (Under Rework, Cleared, Scrapped), `ReworkStationID`	Rework Queue Counter, Defect Disposition Pie Chart, Active Rework Engine Grid
Engine Rework Log (`EngineReworkReport.jsx`)	`/api/trace/engine-rework`	`Prod_Defect_Log` `Prod_Engine_WIP` `Config_SKU` `Config_Model`	`D.EngineNo = W.EngineNo` `W.SKUID = S.SKUID` `S.ModelID = M.ModelID`	`ReworkStartTime`, `ReworkEndTime`, `ReworkActionTaken`, `PartsReplaced`, `ReworkInspector`	Rework Cycle Time KPI, Defect-to-Action Matrix, Detailed Engine Rework Ledger

Module 5: Quality Module

5.1 Architectural Flow Chart

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph UI_Layer["Quality Reports"]
        Q1["DefectReport.jsx (Defects & Pareto)"]
        Q2["PQCAReport.jsx (Audit Monitoring)"]
        Q3["IQC / IPQC / FQC Checklist Reports"]
        Q4["IQC / IPQC / FQC Checkpoint Reports"]
    end

    subgraph API_Layer["Express API Endpoints"]
        E_Q1["GET /api/quality/defect"]
        E_Q2["GET /api/quality/pqca"]
        E_Q3["GET /api/quality/checklist?type=..."]
    end

    subgraph DB_Layer["Database Tables & Procedures"]
        QA1[("Prod_Defect_Log")]
        QA2[("Prod_EnginePlanExecution")]
        QA3[("Prod_Engine_WIP")]
        QA4[("QA_AuditMonitoring")]
        QA5[("Config_AuditList")]
        QA6[("Config_Line")]
        QA7[("Config_Model")]
        QA8[("Config_ModelFamily")]
        QA9[("Config_SKU")]
    end

    Q1 --> E_Q1
    Q2 --> E_Q2
    Q3 --> E_Q3
    Q4 --> E_Q3

    E_Q1 --> QA1 & QA2 & QA3 & QA6 & QA7 & QA8 & QA9
    E_Q2 --> QA4 & QA5 & QA6 & QA7 & QA8
    E_Q3 --> QA4 & QA5 & QA6 & QA7 & QA9
```

5.2 Data Dictionary & Table Cross-Reference

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Defect Report (`DefectReport.jsx`)	`/api/quality/defect`	`Prod_Defect_Log` `Prod_EnginePlanExecution` `Prod_Engine_WIP` `Config_Line` `Config_SKU` `Config_Model`	`D.EngineNo = W.EngineNo` `W.SKUID = S.SKUID` `S.ModelID = M.ModelID` `W.LineID = L.LineID`	`DefectName`, `DefectCategory`, `TotalDefects`, `EngineNo`, `StationID`, DPMO (Defects Per Million Opportunities)	Total Defect Count, DPMO Metric Card, Defect Category Pareto Chart, Defect Log Table
PQCA Report (`PQCAReport.jsx`)	`/api/quality/pqca`	`QA_AuditMonitoring` `Config_AuditList` `Config_Line` `Config_Model` `Config_ModelFamily`	`A.AuditID = AL.AuditID` `A.LineID = L.LineID` `A.ModelID = M.ModelID` `M.ModelFamilyID = F.ModelFamilyID`	`AuditDate`, `Shift`, `AuditorName`, `TotalCheckpoints`, `PassedCheckpoints`, `FailedCheckpoints`, `ComplianceRate`	PQCA Compliance % Gauge, Audit Pass Rate Trend, Non-Conformance Issue Matrix
IQC / IPQC / FQC Checklists (`*ChecklistReport.jsx`)	`/api/quality/checklist?type=IQC\`	IPQC\	FQC`	`QA_AuditMonitoring` `Config_AuditList` `Config_Line` `Config_Model` `Config_SKU`	`A.AuditID = AL.AuditID` `A.LineID = L.LineID` `A.ModelID = M.ModelID` `A.SKUID = S.SKUID`
IQC / IPQC / FQC Checkpoints (`*CheckpointReport.jsx`)	`/api/quality/checklist?type=IQC\`	IPQC\	FQC`	`QA_AuditMonitoring` `Config_AuditList` `Config_Line` `Config_Model` `Config_SKU`	`A.AuditID = AL.AuditID` `A.LineID = L.LineID` `A.ModelID = M.ModelID`

Module 6: Maintenance Module

6.1 Architectural Flow Chart

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph UI_Layer["Maintenance Reports"]
        M1["MaintenanceDashboard.jsx (Overview)"]
        M2["BreakdownReport.jsx (Machine Breakdowns)"]
        M3["DowntimeSummaryReport.jsx (Downtime Analysis)"]
        M4["MTTRMTBFReport.jsx (Reliability Metrics)"]
        M5["PMDashboard.jsx & PMReport.jsx (Preventive Maint)"]
    end

    subgraph API_Layer["Express API Endpoints"]
        E_M1["GET /api/maintenance/dashboard"]
        E_M2["GET /api/maintenance/breakdown"]
        E_M3["GET /api/maintenance/downtime"]
        E_M4["GET /api/maintenance/mttr-mtbf"]
        E_M5["GET /api/maintenance/pm-dashboard & pm-report"]
    end

    subgraph DB_Layer["Database Tables & Procedures"]
        MT1[("Maint_BreakDown_Log")]
        MT2[("Config_Station")]
        MT3[("Config_Line")]
        MT4[("Config_LossCategory")]
        MT5[("Config_User")]
        MT6[("Config_PMSchedule")]
        MT7[("Config_PMCheckList")]
        SP_MDT[["SP: DS_Dashboard_MaintenanceDowntime"]]
    end

    M1 --> E_M1
    M2 --> E_M2
    M3 --> E_M3
    M4 --> E_M4
    M5 --> E_M5

    E_M1 --> MT1 & MT2 & MT3 & MT4
    E_M2 --> MT1 & MT2 & MT3 & MT5
    E_M3 --> SP_MDT
    E_M4 --> MT1 & MT2 & MT3
    E_M5 --> MT6 & MT7 & MT3
```

6.2 Data Dictionary & Table Cross-Reference

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Maintenance Dashboard (`MaintenanceDashboard.jsx`)	`/api/maintenance/dashboard`	`Maint_BreakDown_Log` `Config_Station` `Config_Line` `Config_LossCategory`	`B.StationID = S.StationID` `B.LineID = L.LineID` `B.LossID = LC.LossID`	Total Breakdowns, Active Alarms, Open Work Orders, Total Repair Duration, Station Impact	Open BD Counter, MTTR/MTBF Quick Stats, Station Breakdown Heatmap, Breakdown Severity Chart
Breakdown Report (`BreakdownReport.jsx`)	`/api/maintenance/breakdown`	`Maint_BreakDown_Log` `Config_Station` `Config_Line` `Config_User`	`B.StationID = S.StationID` `B.LineID = L.LineID` `B.TechnicianID = U.UserID`	`BreakdownStartTime`, `BreakdownEndTime`, `Duration`, `FailureMode`, `ActionTaken`, `TechnicianName`, `RootCause`	Breakdown Time Trend, Top Fault Categories Pareto, Machine Breakdown Incident Log

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Downtime Summary Report (`DowntimeSummaryReport.jsx`)	`/api/maintenance/downtime`	`DS_Dashboard_MaintenanceDowntime` (SP)	Param: `@LineID`, `@ProdDate`	`MachineName`, `StationName`, `TotalDownTimeMins`, `BreakdownOccurrences`, Cumulative Loss %	Maintenance Loss Distribution, Station Downtime Pareto, Equipment Downtime Matrix
MTTR & MTBF Report (`MTTRMTBFReport.jsx`)	`/api/maintenance/mttr-mtbf`	`Maint_BreakDown_Log` `Config_Station` `Config_Line`	`B.StationID = S.StationID` `B.LineID = L.LineID`	Calculated $\text{MTTR} = \frac{\sum \text{Repair Time}}{\text{Failures}}$, $\text{MTBF} = \frac{\text{Operating Time}}{\text{Failures}}$, Availability Rate %	MTTR & MTBF Metric Cards, Reliability Trend Line Chart, Station Reliability Ranking Table
PM Dashboard & PM Report (`PMDashboard.jsx`, `PMReport.jsx`)	`/api/maintenance/pm-dashboard` `/api/maintenance/pm-report`	`Config_PMSchedule` `Config_PMCheckList` `Config_Line`	`S.CheckListID = C.CheckListID` `S.LineID = L.LineID`	`ScheduleDate`, `TaskFrequency` (Daily/Weekly/Monthly), `CompletionStatus`, `OverdueDays`, `AssignedTechnician`	PM Compliance % Gauge, Overdue Maintenance Tasks, Scheduled Calendar View, PM Work Order Log

Module 7: Material & Kitting Module

7.1 Architectural Flow Chart

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph UI_Layer["Material & Kitting Reports"]
        MAT1["MaterialDashboard.jsx (Overview)"]
        MAT2["StockStatusReport.jsx & EngineStockReport.jsx"]
        MAT3["MaterialRequestReport.jsx (Andon Calls)"]
        MAT4["KittingDashboard.jsx & KitInspectionReport.jsx"]
        MAT5["KitVsProductionReport.jsx (Kit Gap)"]
        MAT6["MaterialConsumptionReport.jsx (BOM Usage)"]
    end

    subgraph API_Layer["Express API Endpoints"]
        E_MAT1["GET /api/material/dashboard & stock"]
        E_MAT2["GET /api/material/request"]
        E_MAT3["GET /api/material/kitting"]
        E_MAT4["GET /api/material/engine-stock"]
        E_MAT5["GET /api/material/consumption"]
    end

    subgraph DB_Layer["Database Tables & Procedures"]
        S1[("Material_Stock")]
        S2[("Prod_Kit_Inspection_Log")]
        S3[("Prod_Engine_WIP")]
        S4[("Config_Model")]
        S5[("Config_SKU")]
        S6[("Config_ModelFamily")]
        S7[("Config_Line")]
        S8[("Config_PartVariant")]
        S9[("Config_MaterialLocationMapping")]
        S10[("Material_Movement_Log")]
        S11[("Prod_Engine_Part_Geneology")]
    end

    MAT1 --> E_MAT1
    MAT2 --> E_MAT1 & E_MAT4
    MAT3 --> E_MAT2
    MAT4 --> E_MAT3
    MAT5 --> E_MAT3
    MAT6 --> E_MAT5

    E_MAT1 --> S1 & S8 & S9
    E_MAT2 --> S10 & S8
    E_MAT3 --> S2 & S4 & S5 & S7
    E_MAT4 --> S3 & S4 & S5 & S6
    E_MAT5 --> S1 & S8 & S11
```

> [!NOTE] **Bajaj PPMS Data Architecture Note: ERP Integration & Real Tables**

> In the official *Bajaj Pantnagar DB Design*, the column comment *"From SAP_PartMaster"* (e.g. for `Config_PartVariant.PartID`) denotes the **external SAP ERP master data feed** providing part numbers to PPMS. In the physical SQL database `PPMS_BajajPant`, parts and stock are managed through the official tables:

> - `Config_PartVariant`: Complete Part Master table (379+ parts like `ASSEMBLY SHAFT OUTPUT`, `CYLINDER BLOCK`, `CRANKCASE RH/LH`, `PISTON ASSLY`, etc.).

> - `Material_Stock`: Shop-floor physical stock across Store (`StoreQty`), Kit Racks (`LineCKitRackQty`, `LineDKitRackQty`), Lineside buffers (`LinesideCQty`, `LinesideDQty`), and WIP on kits.

> - `Config_MaterialLocationMapping`: Storage area mappings and Min/Max capacity thresholds (`MinAreaCapacity`, `MaxAreaCapacity`).

> - `Material_Movement_Log`: Audit trail of material transfers from Store to Assembly Line.

> - `Prod_Engine_Part_Geneology`: Engine BOM part consumption tracking linking assembled engine serial numbers to component part numbers.

7.2 Data Dictionary & Table Cross-Reference

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Material Dashboard (`MaterialDashboard.jsx`)	`/api/material/dashboard`	`Material_Stock` `Config_PartVariant` `Config_MaterialLocationMapping`	`s.PartID = v.PartID` `s.PartID = loc.PartID`	Total Active Parts, Critical Shortage SKUs (`currentStock <= MinAreaCapacity`), Safety Stock Breaches, Store + Kit Rack + Lineside Inventory	Stock Health Cards, Category Inventory Bar Chart, Critical Shortages Alert Table
Stock Status Report (`StockStatusReport.jsx`)	`/api/material/stock`	`Material_Stock` `Config_PartVariant` `Config_MaterialLocationMapping`	`s.PartID = v.PartID` `s.PartID = loc.PartID`	`PartID`, `PartName`, `AvailableQty` (`StoreQty + LineCKitRackQty + LineDKitRackQty + LinesideCQty + LinesideDQty`), `MinAreaCapacity`, `MaxAreaCapacity`, Status (`Safe`/`Critical`/`Excess`)	Stock Coverage (Days), Inventory Value, Part Master Stock Table
Material Request Report (`MaterialRequestReport.jsx`)	`/api/material/request`	`Material_Movement_Log` `Config_PartVariant`	`m.PartID = v.PartID`	`RequestID` (`REQ-UID`), `PartID`, `PartName`, `SourceAreaID` (Store), `DestinationAreaID` (Line), `Quantity`, `TimeStamp`	Open Andon Calls Counter, Fulfilled vs Pending Chart, Material Dispatch Tracking Grid
Kitting Dashboard & Inspection (`KittingDashboard.jsx`, `KitInspectionReport.jsx`)	`/api/material/kitting`	`Prod_Kit_Inspection_Log` `Config_Model` `Config_SKU` `Config_Line`	`K.SKUID = S.SKUID` `S.ModelID = M.ModelID` `K.LineID = L.LineID`	`KitBarcode`, `TrayID`, `InspectionResult` (Pass/Fail), `MissingPartsCount`, `InspectorID`, Timestamp	Kit Preparation Rate %, Inspection Pass %, Missing Part Pareto, Kit Audit Log Grid
Kit vs Production Report (`KitVsProductionReport.jsx`)	`/api/material/kitting`	`Prod_Kit_Inspection_Log` `Config_Model` `Config_SKU` `Config_Line`	Aggregated by `ProdDate`, `ProdShift`, `ModelID`	Kits Completed vs Engines Assembled, Kit Buffer Gap, Starvation Risk Flag	Kit vs Engine Synchronized Line Chart, Buffer Deficit Warning, Gap Analysis Table
Engine Stock Report (`EngineStockReport.jsx`)	`/api/material/engine-stock`	`Prod_Engine_WIP` `Config_SKU` `Config_Model` `Config_ModelFamily`	`W.SKUID = S.SKUID` `S.ModelID = M.ModelID` `M.ModelFamilyID = F.ModelFamilyID`	`EngineNo`, `ModelName`, `ModelFamilyName`, `Status` (Finished / Cleared / In Stock), Storage Bay	Total Finished Engines, Warehouse Bay Allocation, Stock Aging Breakdown
Material Consumption Report (`MaterialConsumptionReport.jsx`)	`/api/material/consumption`	`Material_Stock` `Config_PartVariant` `Prod_Engine_Part_Geneology`	`s.PartID = v.PartID` `s.PartID = cons.PartID`	`PartID`, `PartName`, `ActualConsumed` (from `Prod_Engine_Part_Geneology`), `ExpectedConsumed`, `VarianceQty`, `VariancePct`	Scrap % Gauge, High-Variance BOM Components, Monthly Consumption Ledger

Module 8: Workforce Module

8.1 Architectural Flow Chart

SPECIFICATION / ARCHITECTURE

```
flowchart TD
    subgraph UI_Layer["Workforce Reports"]
        W1["WorkforceDashboard.jsx (Overview)"]
        W2["AttendanceReport.jsx (Shift Attendance)"]
        W3["SkillMatrixDashboard.jsx & SkillMatrixReport.jsx"]
        W4["WorkforceAllocationReport.jsx (Roster)"]
    end

    subgraph API_Layer["Express API Endpoints"]
        E_W1["GET /api/workforce/dashboard"]
        E_W2["GET /api/workforce/attendance"]
        E_W3["GET /api/workforce/skill-matrix"]
        E_W4["GET /api/workforce/allocation"]
    end

    subgraph DB_Layer["Database Tables & Procedures"]
        WF1[("Config_User")]
        WF2[("Prod_ShiftOperatorAssignment")]
        WF3[("Config_OperatorSkillMapping")]
        WF4[("Config_SkillLevel")]
        WF5[("Config_Skill")]
        WF6[("Config_Line")]
        WF7[("Config_Station")]
    end

    W1 --> E_W1
    W2 --> E_W2
    W3 --> E_W3
    W4 --> E_W4

    E_W1 --> WF1 & WF3 & WF4 & WF2 & WF6 & WF7
    E_W2 --> WF1 & WF2 & WF6
    E_W3 --> WF1 & WF3 & WF4 & WF5 & WF2 & WF6 & WF7
    E_W4 --> WF2 & WF6 & WF7 & WF1
```

8.2 Data Dictionary & Table Cross-Reference

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Workforce Dashboard (`WorkforceDashboard.jsx`)	`/api/workforce/dashboard`	`Config_User` `Prod_ShiftOperatorAssignment` `Config_OperatorSkillMapping` `Config_SkillLevel` `Config_Line` `Config_Station`	`A.UserID = U.UserID` `M.UserID = U.UserID` `M.SkillLevelID = SL.SkillLevelID` `A.LineID = L.LineID`	Total Operators Present, Absenteeism %, Critical Station Skill Coverage, Average Skill Level	Present / Absent StatCards, Skill Tier Distribution Pie, Station Coverage Heatmap
Attendance Report (`AttendanceReport.jsx`)	`/api/workforce/attendance`	`Config_User` `Prod_ShiftOperatorAssignment` `Config_Line`	`A.UserID = U.UserID` `A.LineID = L.LineID`	`OperatorID`, `OperatorName`, `Shift`, `AttendanceStatus` (Present/Absent/On Leave), In Time, Out Time	Shift Attendance % Gauge, Absenteeism Rate Trend, Daily Shift Attendance Roster Table
Skill Matrix Dashboard & Report (`SkillMatrix*.jsx`)	`/api/workforce/skill-matrix`	`Config_User` `Config_OperatorSkillMapping` `Config_SkillLevel` `Config_Skill` `Config_Station`	`M.UserID = U.UserID` `M.SkillID = SK.SkillID` `M.SkillLevelID = SL.SkillLevelID` `SK.StationID = S.StationID`	Operator Competency Rating (L1 Trainee, L2 Semi-Skilled, L3 Independent, L4 Trainer/Expert) per Station	L1-L4 Competency Radar Chart, Operator Skill Matrix Grid, Station Multi-Skilling Index

Report / Component	API Endpoint	Database Tables / Procedures	Joined Columns	Extracted Fields & Metrics	UI Destination
Workforce Allocation Report (`WorkforceAllocationReport.jsx`)	`/api/workforce/allocation`	`Prod_ShiftOperatorAssignment` `Config_Line` `Config_Station` `Config_User`	`A.StationID = S.StationID` `A.LineID = L.LineID` `A.UserID = U.UserID`	`ShiftDate`, `Shift`, `StationName`, `AssignedOperatorName`, Role (Primary / Reliever / Team Leader)	Station Manning Ratio, Unmanned Station Alert, Shift Operator-to-Station Roster Grid

---

4. Master Database Tables Inventory (`PPMS_BajajPant`)

Here is the complete inventory of all database tables and views utilized across the entire application:

A. Production & Shop Floor Execution Tables

1. `dbo.Prod_EnginePlanExecution` : Engine assembly execution records, model/SKU plan vs actual, straight pass, rework, WIP counts.
2. `dbo.Perf_Hourly_OLE` : Hourly operational line efficiency (planned vs total vs good vs rejection quantity).
3. `dbo.Perf_Downtime` : Machine and station downtime stoppages, breakdown durations, categorized losses (8 plant categories).
4. `dbo.Prod_Engine_WIP` : Work-in-progress engine serial tracking across line conveyor buffers.
5. `dbo.Prod_Defect_Log` : Defect entries, quality rejects, defect descriptions, and rework loop triggers.
6. `dbo.Prod_TorqueData_Log` : Fastening data from smart nutrunners (torque Nm, tightening angle, spindle ID, OK/NOK status).
7. `dbo.Prod_Engine_Geneology` : Engine child part scanning, critical component serial numbers, supplier batch trace.
8. `dbo.Prod_Kit_Inspection_Log` : Kitting station verification, parts checklist audit, tray barcode validation.
9. `dbo.Prod_ShiftOperatorAssignment` : Daily shift operator-to-station allocation roster.

B. Quality & Maintenance Execution Tables

10. `dbo.QA_AuditMonitoring` : Shop floor quality audit inspections (PQCA, IQC, IPQC, FQC).
11. `dbo.Maint_BreakDown_Log` : Machine breakdown events, technician logs, root cause analysis, repair times.
12. `dbo.Config_PMSchedule` : Preventive maintenance calendar, scheduled tasks, assigned frequencies.
13. `dbo.Config_PMCheckList` : PM checklist instructions, tool requirements, lubrication schedules.

C. Master & Configuration Tables

14. `dbo.Config_Line` : Production line master (`LineID` , `LineName` , e.g. Line C Engine Assembly).
15. `dbo.Config_Station` : Assembly station master (`StationID` , `StationName` , `StationCode`).
16. `dbo.Config_ModelFamily` : Vehicle / Engine families (e.g. Pulsar, Platina, Dominar).
17. `dbo.Config_Model` : Engine models under families.
18. `dbo.Config_SKU` : SKU codes, BOM variants, and technical specifications.
19. `dbo.Config_LossCategory` : Loss classification categories (Breakdown, Setup, Material, Quality).
20. `dbo.Config_SubLossCategory` : Sub-categories (Mechanical, Electrical, Operator Error, etc.).
21. `dbo.Config_User` : System operators, technicians, auditors, and supervisors.
22. `dbo.Config_AuditList` : Standard audit checkpoints, tolerances, and inspection criteria.
23. `dbo.Config_Skill` : Operational competencies required per station.
24. `dbo.Config_SkillLevel` : L1 to L4 skill competency tier definitions.
25. `dbo.Config_OperatorSkillMapping` : Operator-to-skill matrix evaluation records.
26. `dbo.Config_PartVariant` : Official Part Master (379+ parts) holding `PartID` , `EnginePartID` , `PartName` , and `PartDesc` .
27. `dbo.Material_Stock` : Shop-floor physical stock across Store (`StoreQty`), Kit Racks (`LineCKitRackQty` , `LineDKitRackQty`), Lineside buffers (`LinesideCQty` , `LinesideDQty`), and WIP on kits.
28. `dbo.Config_MaterialLocationMapping` : Storage area mappings and Min/Max capacity thresholds (`MinAreaCapacity` , `MaxAreaCapacity`).
29. `dbo.Material_Movement_Log` : Real-time material transfer and dispatch logs from Store to Assembly Lines.
30. `dbo.Prod_Engine_Part_Geneology` : Engine BOM part consumption genealogy linking assembled engine serial numbers to component part numbers.

D. Stored Procedures Executed

1. `dbo.DS_Dashboard_ProductionKPI` : Calculates shift target, actual produced, straight pass, WIP, and rollover.
2. `dbo.GV_ProdShiftPlan` : Returns the official production shift plan authored for the shop floor.

3. `dbo.DS_Dashboard_ProcessConveyor` : Fetches real-time conveyor parameters, VFD speed, and JPH.
4. `dbo.DS_Dashboard_TrackTraceWIP` : Aggregates buffer WIP levels and bottleneck status across all line stations.
5. `dbo.DS_Dashboard_MaintenanceDowntime` : Computes maintenance loss breakdown and downtime durations.